Laborator 12

12.1 Thread

Firele de executie (threads) permit executia concurenta de cod. In general timpul de executie este mai mic, se utilizeaza mai bine resursele sistemului, aplicatia este mai responsiva.

Multithreading se foloseste in:

- Webserver
- Aplicatii GUI
- Jocuri
- Sistem real-time
- Procesari inlantuite de date (Data processing pipelines)

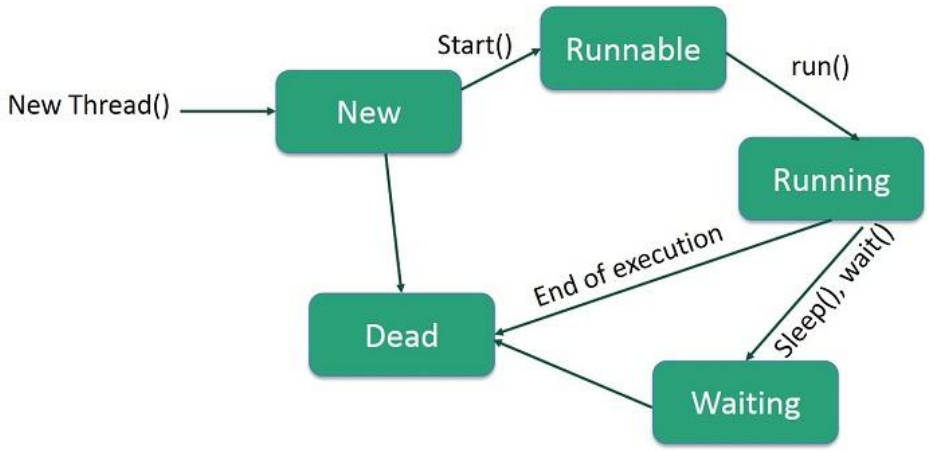
Tot ceea ce este nevoie pentru un fir de executie se afla in clasa Thread.

Java are suport pentru multithreading cu ajutorul clasei *Thread*, interfata *Runnable* si clasele din pachetul *java.util.concurrent.* Prin aceasta putem rula sarcini in paralel in cadrul aceleiasi aplicatii.

Un fir de executie ruleaza o bucata de cod in mod independent, si utilizeaza memorie si resurse sistem la comun cu alte fire de executie ale aplicatiei/procesului in care fac parte.

Fiecare thread are urmatorul clicu de viata:

New > Runnable > Running > Blocked / Waiting > Terminated



Ciclul de viata este gestionat de JVM si sistemul de operare, pentru ca instanta Thread utilizeaza thread nativ (al sistemului de operare).

Cu ajutorul metodelor *sleep*(), *join*(), *yield*() si *setDaemon*() realizam controlul executiei intre threaduri

Pentru evitarea modificarii concomitente (race condition) a datelor accesibile de mai multe fire de executie vom folosi *sincronizarea*. Pentru gestiunea sincronizarii folosim *synchronized*, *wait*() sau *notify*().

Laborator 12

12.1.1 Instantierea unui nou Thread prin implementarea interfetei Runnable

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start(); // Start the thread
    }
}
```

12.1.2 Instantierea unui nou Thread prin mostenirea clasei Thread

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

12.1.3 Instantierea unui nou Thread utilizand expresii lambda

```
public class LambdaThread {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> {
            System.out.println("Thread running with lambda!");
        });
        thread.start();
    }
}
```

Exemplu de utilizare doua thread-uri:

```
class Task1 extends Thread {
    public void run() {
        System.out.println("Task1 is running");
    }
}
```

```
class Task2 extends Thread {
    public void run() {
        System.out.println("Task2 is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Task1 t1 = new Task1(); // Create an instance of Task1
        Task2 t2 = new Task2(); // Create an instance of Task2
        t1.start(); // Start Task1
        t2.start(); // Start Task2
    }
}
```

12.1.3 Introducerea de pauze

Cu ajutorul metodei sleep() introducem o pauza executiei firului.
Cu ajutorul metodei join() asteptam ca threadul sa se termine
Cu ajutorul metodei yield() sugereaza ca threadul curent doreste sa fie pauzat
Cu ajutorul metodei interrupt() inregistram cererea ca threadul sa fie oprit

12.1.4 Race condition

Apare cand 2 sau mai multe thread-uril acceseaza pt citire sau scriere aceeasi variabila sau zona de memorie.

```
public class CounterExample {
    static int count = 0;

    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) count++;
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) count++;
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final count: " + count); // Output may vary!
    }
}
```

12.1.5 Sincronizarea si controlul accesului concurrent

In Java, sincronizarea este construita pe baza unei entitati interne numita ‘intrinsic lock’ sau ‘monitor lock’. Prin acestea se asigura acces exclusiv la instanta. Fiecare obiect are asociat un ‘intrinsic lock’. Cand un thread are nevoie de acces consistent, trebuie sa obtina aceste lock-uri si cand terimina sa elibereze lock-urile. Cat timp un lock este in proprietatea unui thread, niciun alt thread nu poate obtine simultan acest lock.

Obiectele si variabilele utilizabile de mai multe threaduri necesita acces sincronizat utilizand cuvantul cheie *synchronized*. Acesta se poate aplica fie metodelor fie bloc-urilor:

```
public synchronized void increment() {
    count++;
}

public void increment() {
    synchronized (this) {
        count++;
    }
}
```

Sincronizare in context static:

```
public static synchronized void staticIncrement() {
    // synchronized at the class level
}

public void staticIncrement() {
    synchronized (CounterExample.class) {
        count++;
    }
}
```

Se recomanda sincronizarea doar a liniilor de cod critice.

12.1.6 Comunicare intre thread-uri folosind *wait()* si *notify()*

Metoda wait() renunta la lock si pune threadul in stare de asteptare. Iese din aceasta stare doar cand un alt thread apeleaza *notify*() sau *notifyAll*() pentru acest thread.

```
class SharedData {
    private boolean available = false;

    public synchronized void produce() throws InterruptedException {
        while (available) {
            wait(); // Wait until item is consumed
        }
        System.out.println("Producing item...");
        available = true;
        notify(); // Notify waiting consumer
```

```
    }

    public synchronized void consume() throws InterruptedException {
        while (!available) {
            wait(); // Wait until item is produced
        }
        System.out.println("Consuming item...");
        available = false;
        notify(); // Notify waiting producer
    }
}
```

12.1.7 Cuvantul cheie *volatile*

In mod implicit, modificarile realizate de un thread nu sunt neaparat vizibile altor thread-uri din cauza cache-ului din microprocessor, ori optimizari.
Cuvantul cheie volatile precizeaza masinii virtuale ca trebuie tot timpul sa citeasca din memoria sistemului acea variabila:

```
class FlagExample {
    private volatile boolean running = true;

    public void stop() {
        running = false;
    }

    public void run() {
        while (running) {
            // do work
        }
    }
}
```

12.1.8 Erori frecevente

a) Apel al metodei *run*() si nu *start*()

```
Thread t = new Thread(() -> doWork());
t.run();   // runs on the main thread
t.start(); // runs in a new thread
```

b) Netratarea corecta a exceptiei InterruptedException:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Don't just ignore it
}
```

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // Restore interrupt status
}
```

c) Crearea de prea multe thread-uri
Crearea prea multor instante Thread, consuma CPU pentru schimbare de context excesiv, sau poate determina aruncarea OutOfMemoryException, consum mare de resurse.

12.2 Parallel Computing

Parallel computing se refera la procesul de impartire a unui set mare de date in submultimi care pot fi executate simultan pe nucleele din processor. Tinta este reducerea timpului de executie.
Este frecvent utilizata in

- Sarcini bazate pe functionalitati processor
- Operatii pe date paralele
- Procesare in batch, sau algoritmi fork/join

Feature	Multithreading	Parallel Computing
Primary Goal	Improve responsiveness and task coordination	Increase speed through simultaneous computation
Typical Use Case	I/O-bound or asynchronous tasks	CPU-bound or data-intensive workloads
Execution Model	Multiple threads, possibly interleaved on one core	Tasks distributed across multiple cores or processors
Concurrency vs. Parallelism	Primarily concurrency (tasks overlap in time)	True parallelism (tasks run at the same time)
Thread Communication	Often requires synchronization	Often independent tasks (less inter-thread communication)
Memory Access	Threads share memory	May share or partition memory
Java Tools & APIs	Thread, ExecutorService, CompletableFuture	ForkJoinPool, parallelStream(), and ExecutorService configured for CPU-bound tasks
Performance Bottlenecks	Thread contention, deadlocks, synchronization latency	Poor task decomposition, load imbalance
Scalability	Limited by synchronization and resource management	Limited by number of available CPU cores
Determinism	Often non-deterministic due to timing and order	Can be deterministic with proper design

Este utilizata la:

Laborator 12

- Procesare de imagine si video
- Simulari matematice
- Analiza de date in cantitati mari
- Operatii pe vectori sau matrici
- Citire si conversie de fisiere in mod batch

Ex1: afiseaza numarul de numere prime in intervalul 1…500000

```
public class PrimeNum {

        public static void main(String[] args) {
            System.out.println("Start");
            long t1 = System.currentTimeMillis();
            long count = Stream.iterate(0, n -> n + 1)
                    .limit(500_000)
                    .parallel()                 //cu 6s, fara linia asta  26s
                    .filter(PrimeNum::isPrime)
                    //.peek(x -> System.out.format("%s\t", x))
                    .count();
            long t2 = System.currentTimeMillis();
            System.out.println("\nTotal: " + count +"  time: "+ (t2 - t1));

        }

        public static boolean isPrime(int number) {
            if (number <= 1) return false;
            return IntStream.rangeClosed(2, number / 2).noneMatch(i -> number % i == 0);
        }
}
```

Ex2: in loc de ‘Hello GEEKS’ afisare aleatorie a literelor GEEKS

```
class ParallelStreamExample {
    public static void main(String[] args)
    {
        // create a list
        List<String> list = Arrays.asList("Hello ",
                          "G", "E", "E", "K", "S!");

        // using parallelStream()
        // method for parallel stream
        list.parallelStream().forEach(System.out::print);
    }
}
```

```
Output:  ES!KGEHello
```

12.3 Probleme

12.3.1 Deschideti proiectul ProiectareSoftware si adaugati sursele din fisierul *Laborator12_starter.zip*
a) Convertiti clasa CookingTasks in Thread, si porniti cele 4 threaduri declarate in functia main() Rezultatul este ca taskurile se vor executa acum intercalat de la cele 4 thread-uri.

b) Rezolvati race-condition ce apare pe campul *usedWater* din clasa Restaurant

Comiteti sursele in GitHub

12.4 Referinte

Multithreading in Java: Concepts, Examples, and Best Practices
https://www.digitalocean.com/community/tutorials/multithreading-in-java

Java - Multithreading
https://www.tutorialspoint.com/java/java_multithreading.htm

Multithreading in Java
https://www.geeksforgeeks.org/java/multithreading-in-java/

A Beginner's Guide to Multithreading in Java(Part 1)
https://medium.com/@pratik.941/a-beginners-guide-to-multithreading-in-java-part-1-5a4834b04693

Threads and Multithreading in Java
https://cse.iitkgp.ac.in/~dsamanta/java/ch6.htm

Java 8 Parallel Streams Examples
https://mkyong.com/java8/java-8-parallel-streams-examples/

Parallelism
https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html

When to Use a Parallel Stream in Java
https://www.baeldung.com/java-when-to-use-parallel-stream

Custom Thread Pools in Java - Parallel Streams
https://www.baeldung.com/java-8-parallel-streams-custom-threadpool

Hints
12.3.1
- CookingTasks ar trebui sa mosteneasca clasa Thread
- Implementati metoda run()

12.3.2 utilizati cuvantul cheie synchronized